

Relazione di progetto — ConversAI

Corso di Applicazioni Web — A.A. 2025/2026

[Nome Cognome] — matricola [numero di matricola]

1. Introduzione

ConversAI è un'applicazione web che simula un'interfaccia di conversazione con un assistente, organizzata in progetti e conversazioni. È realizzata come un'unica applicazione Express.js (Node.js) con persistenza su SQLite e un frontend in HTML/CSS/JavaScript "vanilla", senza framework front-end, seguendo lo schema presentato a lezione negli esempi del corso (AA_static-server, AE_create-task, AF_persistent-tasks). Implementa i requisiti 4.1-4.4 del testo del progetto: gestione CRUD di progetti e conversazioni, invio e recupero messaggi, e una funzione di simulazione della risposta dell'assistente basata su regole.

2. Architettura dell'applicazione

Il progetto è un singolo progetto Express, non due processi separati per frontend e backend: **server.js** serve sia i file statici del frontend (con `express.static("public")`) sia le rotte API dinamiche, definite direttamente sull'oggetto `app` (`app.get/post/put/delete`) senza `express.Router()`, che il corso non mostra mai. Questa è la struttura di tutti gli esempi delle slide: una cartella `public/` con i file statici e un unico file di rotte.

L'unica separazione in più file adottata è quella mostrata esplicitamente a lezione nella slide sulla persistenza SQLite: **database.js** come "storage layer" a parte, in modo che `server.js` si occupi solo di HTTP (rotte, status code, JSON) e `database.js` solo di SQLite (connessione, creazione tabelle, query). Le rotte in `server.js` chiamano le funzioni esportate da `database.js`, che comunicano il risultato tramite un ultimo parametro `handleResult(error, dati)`, lo stesso schema a callback error-first mostrato per il modulo `sqlite3`.

Struttura delle cartelle:

```
conversai/  
  data/                -> conversai.sqlite, creato in automatico, non versionato  
  public/              -> index.html, style.css, app.js: tutto il frontend, servito  
staticamente  
  utils/simulator.js   -> funzione di simulazione della risposta assistente  
  database.js          -> connessione SQLite, creazione tabelle, funzioni di storage  
  init-db.js           -> script eseguibile per inizializzare/seedare il database  
  server.js            -> punto di ingresso: tutte le rotte API sono qui, dirette  
  package.json, .gitignore, README.md
```

Dipendenze: solo due, dichiarate in `package.json` (`express` per il server e le rotte, `sqlite3` per la persistenza). Non è stata usata alcuna libreria di validazione, ORM o framework front-end: gli unici strumenti sono quelli mostrati a lezione. Tre script npm: `start` (avvio normale), `dev` (avvio con `node --watch`, riavvio automatico su modifica dei file, nativo da Node 18+, senza bisogno di `nodemon`) e `init-db` (esegue `init-db.js`).

3. Struttura del database

La persistenza usa SQLite tramite il pacchetto `sqlite3` (asincrono, a differenza di `better-sqlite3` che è sincrono e non è la libreria mostrata a lezione). Le tre tabelle sono create con `CREATE TABLE IF NOT EXISTS` all'avvio del server, così l'applicazione "si auto-inizializza" anche senza eseguire nulla a mano; in aggiunta è previsto uno script `init-db.js` dedicato per creare il database prima del primo avvio e inserire un progetto di esempio.

Tabella	Colonne principali	Vincoli
projects	id, nome, descrizione, data_creazione, data_aggiornamento	id chiave primaria; nome obbligatorio
conversations	id, project_id, topic, data_creazione, data_aggiornamento	FK project_id -> projects(id) ON DELETE CASCADE
messages	id, conversation_id, ruolo, contenuto, data_creazione	FK conversation_id -> conversations(id) ON DELETE CASCADE; CHECK ruolo IN ('user', 'assistant')

Le date sono salvate come TEXT in formato ISO 8601 (`new Date().toISOString()`), perché SQLite non ha un tipo datetime nativo e il formato ISO è ordinabile alfabeticamente come lo sarebbe cronologicamente. Le foreign key con ON DELETE CASCADE delegano al database il compito di eliminare conversazioni e messaggi collegati quando si elimina un progetto, invece di farlo con query multiple lato applicazione: per attivare questo comportamento è necessario eseguire esplicitamente `PRAGMA foreign_keys = ON`, perché SQLite lo disabilita di default per compatibilità storica.

Scelte nelle funzioni di storage: le funzioni `createProject`, `createConversation` e `createMessage` costruiscono l'oggetto di risposta direttamente in JavaScript, usando `this.lastID` (l'id assegnato da SQLite) più i valori già noti dall'input, senza una seconda query di riletture: è lo stesso schema dell'esempio `createTask` mostrato a lezione, dove dopo l'INSERT si conoscono già tutti i campi della riga appena creata. Le funzioni di aggiornamento (`updateProject`, `updateConversation`) restano invece basate su una riletture (`getProjectById`/`getConversationById`) dopo l'UPDATE, perché quelle funzioni conoscono solo i campi che stanno modificando e non `data_creazione`: senza riletture non potrebbero restituire l'oggetto completo richiesto dalla rotta.

4. Endpoint REST progettati

Gli endpoint distinguono collezione (`/api/projects`), singola risorsa (`/api/projects/:id`) e sotto-collezione annidata (`/api/projects/:id/conversations`). Sono implementati PUT e DELETE su entrambe le risorse principali (progetti e conversazioni), oltre al minimo richiesto dal testo. I messaggi non hanno un endpoint di primo livello perché non ha senso accedervi isolatamente, solo "i messaggi di questa conversazione".

Metodo	Path	Descrizione	Status
GET	<code>/api/projects</code>	Elenco progetti	200
POST	<code>/api/projects</code>	Crea progetto {nome, descrizione?}	201 / 400
GET	<code>/api/projects/:id</code>	Dettaglio progetto	200 / 404
PUT	<code>/api/projects/:id</code>	Modifica progetto {nome, descrizione?}	200 / 400 / 404
DELETE	<code>/api/projects/:id</code>	Elimina progetto (a cascata)	204 / 404
GET	<code>/api/projects/:id/conversations</code>	Conversazioni di un progetto	200 / 404
POST	<code>/api/projects/:id/conversations</code>	Crea conversazione {topic}	201 / 400 / 404
GET	<code>/api/conversations/:id</code>	Dettaglio conversazione	200 / 404
PUT	<code>/api/conversations/:id</code>	Modifica topic {topic}	200 / 400 / 404
DELETE	<code>/api/conversations/:id</code>	Elimina conversazione (a cascata)	204 / 404
GET	<code>/api/conversations/:id/messages</code>	Messaggi di una conversazione	200 / 404
POST	<code>/api/conversations/:id/messages</code>	Invia messaggio {contenuto} + risposta simulata	201 / 400 / 404

5. Flusso di comunicazione front-end / back-end

Il ciclo è sempre lo stesso: click o submit nel frontend -> `fetch()` verso l'endpoint corrispondente -> Express riceve, valida `req.body`, interroga SQLite tramite `database.js` -> risposta JSON con status code coerente -> il frontend legge la risposta e aggiorna il DOM, senza mai ricaricare la pagina. Il frontend usa esclusivamente catene di `.then()/catch()`, non `async/await`: è lo stesso stile mostrato in tutti gli esempi del corso. Una funzione di supporto, `leggiRisposta`, centralizza il controllo dello status HTTP (equivalente al `"checkStatus/readJsonOrThrow"` visto a lezione), perché `fetch()` non genera da solo un errore per gli status 4xx/5xx.

Il DOM si aggiorna sempre con `document.querySelector('.classe')` (mai `getElementById`) e con `elemento.replaceChildren()` per svuotare le liste prima di ridisegnarle, come mostrato a lezione. Il flusso più articolato è l'invio di un messaggio: il backend salva prima il messaggio dell'utente, genera la risposta simulata e la salva come secondo messaggio, aggiorna la data dell'ultima modifica della conversazione, e restituisce entrambi i messaggi in un'unica risposta (`{ messaggioUtente, messaggioAssistente }`): il frontend li aggiunge subito al DOM senza dover rifare una GET per rileggerli.

6. Funzione di simulazione della risposta assistente

La funzione `generateAssistantReply(userMessage, topic)` in `utils/simulator.js` genera una risposta "finta" con semplici controlli su stringhe (if/else e template string), senza alcuna intelligenza artificiale reale, come richiesto esplicitamente dal testo del progetto. Le regole sono valutate in ordine e la prima che risulta vera determina la risposta: (1) saluto iniziale ("ciao", "salve", ecc.) -> risposta di benvenuto contestualizzata al topic; (2) presenza della parola "grazie" -> risposta di cortesia; (3) presenza di "aiuto"/"help" -> richiesta di maggiori dettagli; (4) messaggio che termina con "?" -> riconosciuto come domanda, con eco del testo ricevuto; (5) caso di default -> eco del messaggio (troncato a 60 caratteri se più lungo), contestualizzato al topic della conversazione.

7. Problematiche principali ed errori riscontrati

Durante lo sviluppo sono incappato in tre problemi concreti che mi hanno fatto capire meglio come funziona davvero lo stack usato a lezione, non solo a livello di sintassi.

7.1 Commento scritto dentro una stringa SQL. Avevo aggiunto una nota per me stesso direttamente dentro la stringa passata a `database.run()`: `data_creazione TEXT NOT NULL, //non esiste un tipo data`. Avviando il server ottenevo `SQLITE_ERROR: near "//": syntax error`. La causa: tutto ciò che sta dentro i backtick passati a `database.run()` è testo SQL letterale, non JavaScript, e SQL non riconosce i commenti `//` (usa `--`). Soluzione: ho spostato il commento fuori dalla stringa, come riga JavaScript normale prima della chiamata.

7.2 Server che non si metteva in ascolto. Il processo `node server.js` restava avviato (tenuto vivo dalla connessione SQLite aperta) ma `curl` restituiva "Failed to connect". Il file `server.js` definiva tutte le rotte ma non terminava mai con `app.listen(port, ...)`: senza quella chiamata, Express non apre mai nessuna porta in ascolto. Soluzione: aggiunta la parte finale mancante (`express.static("public")`, gestore 404 su `/api` e infine `app.listen()`).

7.3 "No such table" nello script di inizializzazione. Eseguendo `npm run init-db` ottenevo saltuariamente `SQLITE_ERROR: no such table: projects`, anche se la `CREATE TABLE` era corretta. La causa è che il pacchetto `sqlite3`, senza `serialize()`, non garantisce che i comandi vengano eseguiti nell'ordine in cui vengono chiamati: `init-db.js` fa `require("./database")` e subito dopo interroga le tabelle, quindi la query poteva partire prima che la creazione delle tabelle fosse completata. Soluzione: ho chiamato `database.serialize()` subito dopo l'apertura della connessione, che mette la connessione in modalità seriale in modo permanente, così ogni comando aspetta che il precedente sia finito, indipendentemente da quale file lo chiama.

8. Funzionalità aggiuntive e scelte progettuali motivate

Oltre al minimo richiesto (requisiti 4.1-4.4, lavorando da solo) ho implementato PUT e DELETE su entrambe le risorse principali, non solo su una. La validazione dei dati in ingresso non è stata estratta in un modulo condiviso: è scritta per esteso in ogni rotta, in due passi (controllo del tipo, poi trim e controllo di stringa vuota), esattamente come mostrato nell'esempio AF_persistent-tasks a lezione — nessuna slide del corso mostra una funzione di validazione riutilizzabile. Per lo stesso motivo non è stata inclusa la dipendenza cors: frontend e backend sono serviti dallo stesso processo Express sulla stessa origine, quindi non c'è mai una richiesta cross-origin da abilitare.

L'interfaccia grafica è stata rifinita oltre il minimo funzionale: è responsive (sidebar a scomparsa sotto i 992px, sempre visibile sopra), con un menu a fisarmonica progetto -> conversazioni, form di creazione/modifica al posto dei semplici `window.prompt()`, e uno stile scuro con colore d'accento personalizzato. Questa parte usa Bootstrap 5, una scelta personale motivata dalla velocità di ottenere un'interfaccia curata e responsive: va segnalato che nessun esempio del corso usa Bootstrap (è citato solo una volta nella teoria, come esempio generico di framework di layout), quindi è una scelta ulteriore rispetto allo stile "vanilla" degli esempi delle slide, non una tecnica mostrata a lezione.

Limitazioni note

Il simulatore di risposte non è un vero sistema di intelligenza artificiale: è deliberatamente così, come richiesto dal testo del progetto, e non un limite tecnico da correggere. Non è implementata alcuna autenticazione o gestione multi-utente (tutti i progetti sono visibili a chiunque apra l'app): non è richiesta dal testo, che descrive un'applicazione a singolo utente. Non sono presenti test automatizzati: il testo del progetto non li richiede tra i requisiti minimi 4.1-4.4.

9. Conclusione

ConversAI copre tutti i requisiti richiesti (persistenza SQLite, API REST con validazione e codici di stato coerenti, frontend dinamico senza reload, simulazione della risposta dell'assistente) restando il più possibile aderente allo stile e ai pattern mostrati a lezione per le parti backend, con alcune estensioni personali e motivate soprattutto sul fronte dell'interfaccia utente.